Spring Core + Spring Boot Internals – Interview Questions

1. Why did Spring come into the picture when we already had JDBC and Servlets?

Before Spring, enterprise Java was tightly coupled, hard to test, and full of boilerplate code. Developers manually created objects, managed transactions, handled connections, and wrote repetitive error handling.

Any small change forced changes across many classes. Spring solved this by managing objects for us and wiring them automatically.

It introduced inversion of control, so business logic no longer creates its dependencies.

2. What problem does IoC solve in real applications?

IoC removes the responsibility of object creation from business classes. Without IoC, your service classes would create repositories using new, which tightly binds them to implementations.

With IoC, the container creates objects and injects them where needed. This allows you to switch implementations, mock dependencies for testing, and reduce coupling.

3. What is the real difference between IoC and DI?

IoC is the **principle** — the control of object creation is inverted to the container.
DI is the **mechanism** — the way dependencies are actually provided.
You follow IoC by letting Spring manage objects.
You apply DI when Spring injects those objects into your classes using constructors or setters.
So IoC is the design idea, and DI is the technical execution of that idea.

4. How does Spring create and manage objects internally?

When the application starts, Spring scans class paths, reads configurations, and identifies beans.

It creates a container called the **Application Context**. This container instantiates beans, resolves dependencies, applies lifecycle callbacks, and stores them in memory. Whenever a bean is requested, Spring either returns an existing instance or creates one based on scope. From this point, Spring manages the entire object lifecycle.

5. What happens from application start to first API request?

Spring Boot starts → loads configurations → creates Application Context (IoC Container)→ scans beans → performs dependency injection → initializes web server → maps controllers.
When a request comes, it passes through the embedded Tomcat Server → Dispatcher Servlet → Handler Mapping → Controller → Service → Repository → Database.
The response flows back through the same layers to the client.
Spring manages this entire flow automatically.

6. What is a Spring Bean?

A Spring Bean is just a normal Java object whose lifecycle is controlled by Spring.
Spring creates it, injects its dependencies, manages its scope, and destroys it when the application stops.
You never create a bean using new in business logic.
Once it is a bean, Spring tracks it and wires it wherever needed.

7. What are the major phases of the bean lifecycle?

Spring first instantiates the bean → injects dependencies → calls initialization methods → makes it available for use → When the application shuts down, it calls destroy methods.

You can hook into this lifecycle using @PostConstruct, Initializing Bean, and @PreDestroy.

8. Why is singleton scope not truly “single” in real systems?

Spring singleton means one instance per container, not one per JVM or application cluster.
In microservices, each service instance has its own container, so each one gets its own singleton.
Also, if you run multiple application contexts, each has its own singleton beans.

9. When should we use prototype scope?

Use prototype when you need a new object for every request and the object is stateful.
Spring will not manage the full lifecycle after creation. This is useful for request-specific data holders or temporary processors. But you must manually handle cleanup.

10. Difference between @Component, @Service, and @Repository?

They all create beans, but their intent is different.
@Service indicates business logic.
@Repository represents data access and adds exception translation.
@Component is a generic bean.
This semantic clarity helps both developers and Spring internally.

11. Why developers prefer constructor injection over Setter/Field Injection?

Constructor injection ensures all required dependencies are present at object creation time. It makes the class immutable, easy to test, and prevents Null Pointer Exceptions. It also avoids circular dependencies and allows Spring to detect missing beans early during startup.

12. Why is field injection discouraged?

Field injection hides dependencies and makes testing difficult because you cannot easily inject mocks. It also breaks immutability and makes classes harder to reason about. Constructor injection clearly shows required dependencies and allows easier unit testing.

13. Difference between @Configuration and @Component?

@Configuration is a special type of Component that creates proxy beans for @Bean methods. It ensures that each @Bean method returns the same instance when called. @Component does not apply this behaviour. So, use @Configuration for config classes.

14. @Bean vs Component scanning – when to use what?

Use @Bean when the class is from a third-party library or when you need custom creation logic.

Use component scanning when the class is part of your codebase and can be annotated directly.

15. What is circular dependency and why is it dangerous?

Circular dependency happens when two beans depend on each other. Spring cannot decide which one to create first, causing startup failure. It also indicates poor design and tight coupling. Constructor injection prevents this at compile time.

16. What does Spring Boot auto-configure?

It configures Data Source, Entity Manager, Tomcat, Jackson, Logging, Security default etc.

It reads your classpath and properties and enables features automatically. This saves hundreds of lines of XML or Java config.

17. How does auto-configuration work internally?

Spring Boot uses spring.factories to load auto-config classes.
Each auto-config is condition-based using @ConditionalOnClass, @ConditionalOnProperty.
If conditions match, Spring creates beans automatically.

18. application.yml vs application.properties – real difference?

Both do the same job. YAML is hierarchical and cleaner for complex configs. Properties is flat and simpler. YAML is easier for microservices with nested configs.

19. What are profiles used for?

Profiles allow environment-specific configuration.
You can have dev, test, prod configs and switch without code changes.
This avoids hardcoding environment logic.

20. How does embedded Tomcat change architecture?

Earlier, we deployed WAR to server.
Now, server is inside the app.
This allows containerization, microservices, and fast deployments.

21. What is DispatcherServlet and why is it called the front controller?

DispatcherServlet is the single-entry point for all HTTP requests in a Spring Boot application. Every request first reaches this servlet before going to any controller. It decides which controller method should handle the request, executes it, and then returns the response. Because it controls the full request flow, it is called the "front controller." This design centralizes routing, exception handling, and response processing in one place.

22. What happens internally when a REST API is called?

The request comes to embedded Tomcat Server → which forwards it to DispatcherServlet. → DispatcherServlet checks HandlerMapping to find the matching controller method → It then uses HandlerAdapter to call that method → The controller returns a response object → which is converted into JSON by Jackson → DispatcherServlet then sends the response back through Tomcat to the client.

23. Why is layered architecture important in Spring applications?

Layered architecture separates concerns between Controller, Service, and Repository layers. Controllers handle HTTP logic, Services handle business rules, and Repositories handle database access. This separation keeps code clean, testable, and scalable. It also allows changes in one layer without breaking others, which is essential in large enterprise projects.

24. What is @ControllerAdvice and why is it powerful?

@ControllerAdvice is a global error-handling mechanism. Instead of writing try-catch blocks in every controller, you define centralized exception handlers. Whenever an exception occurs, Spring routes it to the matching method in the advice class. This makes error handling consistent, clean, and easy to maintain.

25. Difference between @ExceptionHandler and @ControllerAdvice?

@ExceptionHandler handles exceptions only inside the same controller.

@ControllerAdvice handles exceptions globally across all controllers.

@ControllerAdvice avoids duplicated error-handling logic and keeps controllers clean.

26. Why is logging important in Spring Boot applications?

Logging helps you understand what is happening inside your application.

It is used for debugging, monitoring, and auditing. Without logs, production issues are nearly impossible to diagnose.

Logging should show flow, errors, and system state — not sensitive data or noise.

27. What should we log and what should we avoid?

You should log request flow, errors, important state changes, and system events.

You should avoid logging passwords, tokens, personal data, and large payloads.

Over-logging slows the system and makes logs useless. Good logs are meaningful and minimal.

28. What is a common Spring Boot startup failure cause?

The most common cause is missing or misconfigured beans. Circular dependencies, wrong package scanning, missing database config, and incorrect properties can also cause startup failure.

Spring usually prints the root cause clearly in the stack trace.

29. What does "No qualifying bean found" mean?

It means Spring tried to inject a dependency but could not find a matching bean in the context.

This usually happens when a class is not annotated, not scanned, or multiple beans exist without qualifiers.

30. Why does circular dependency fail with constructor injection?

Constructor injection needs all dependencies before object creation. If two classes depend on each other, Spring cannot create either first, so it fails at startup.

This forces you to redesign and remove bad coupling.

31. How does Spring manage memory for beans?

Spring stores singleton beans in the ApplicationContext cache.

They live if the application runs.

Prototype beans are created and forgotten by Spring after injection. Developers must manage their cleanup manually.

32. What is @PostConstruct used for?

@PostConstruct runs after dependency injection is completed. It is used for initialization logic like loading caches, validating configuration, or opening connections. It runs only once per bean.

33. What is @PreDestroy?

@PreDestroy runs before the bean is removed from the container. It is used to close resources, connections, or threads. This prevents memory leaks and resource issues.

34. What is a Spring Context?

The Spring Context is the container that stores all beans, manages dependencies, and controls lifecycle. Everything in Spring works inside this context. Without it, there is no dependency injection.

35. What is BeanDefinition?

BeanDefinition is Spring's internal metadata about a bean. It contains class name, scope, dependencies, and lifecycle methods. Spring uses this information to create and manage beans.

36. What is ApplicationContext vs BeanFactory?

BeanFactory is the basic container. ApplicationContext is the advanced version that supports events, internationalization, and auto-configuration.

Spring Boot always uses ApplicationContext.

37. Why is Spring considered loosely coupled?

Because classes do not create or control their dependencies. They only declare what they need. Spring injects implementations at runtime.

This allows easy replacement, testing, and scaling.

38. What is dependency graph in Spring?

It is the internal map that shows which beans depend on others.

Spring builds this graph during startup to decide creation order and detect circular references.

39. Why is component scanning important?

Component scanning allows Spring to automatically detect and register beans. Without it, you would have to manually define every bean. ComponentScan reduces configuration and speeds development.

40. What happens if two beans of same type exist?

Spring will throw an ambiguity error unless you use @Primary or @Qualifier to tell which one to inject.

41. What is @Primary?

@Primary marks one bean as default when multiple candidates exist.

Spring uses it unless a qualifier is specified.

42. What is @Qualifier?

@Qualifier tells Spring exactly which bean to inject when multiple beans exist.

It removes ambiguity.

43. What is a configuration class proxy?

Spring creates a proxy for @Configuration classes to ensure that @Bean methods return the same singleton instance. This avoids multiple object creation.

44. Why is Spring Boot faster to develop than Spring?

Because it removes XML, provides defaults, auto-configures infrastructure, embeds server, and reduces setup time. Developers focus on business logic instead of configuration.

45. What is convention over configuration?

Spring Boot assumes default settings, so you don't need to define everything. You override only what you need. This reduces complexity and speeds development.

46. How does Spring handle exceptions internally?

Exceptions bubble up to DispatcherServlet, which checks for matching handlers in @ControllerAdvice and returns formatted responses.

47. Why is configuration externalization important?

It allows changing environment behaviour without code changes.

This is critical for CI/CD and cloud deployments.

SPRING & SPRING BOOT ANNOTATIONS

Annotation	Category	Where Used	What it Does (Simple Explanation)
@SpringBootApplication	Boot Core	Main class	Combination of @Configuration, @EnableAutoConfiguration, and @ComponentScan. Starts Spring Boot app.
@Configuration	Core	Config class	Marks class as Spring configuration and enables proxying for @Bean methods.
@ComponentScan	Core	Config class	Tells Spring where to search for beans.
@EnableAutoConfiguration	Boot	Main class	Enables Spring Boot auto-configuration based on classpath.
@Component	Core	Any class	Generic Spring bean.
@Service	Core	Service layer	Business logic bean.
@Repository	Core	DAO layer	Database access bean with exception translation.
@Controller	Web	Controller layer	MVC controller for returning views.
@RestController	Web	API layer	Combines @Controller + @ResponseBody for REST APIs.
@Autowired	DI	Fields/Constructors	Injects dependency automatically.
@Qualifier	DI	Injection point	Specifies which bean to inject when multiple exist.

@Primary	DI	Bean class	Marks one bean as default.
@Bean	Core	Config class	Creates and manages a bean manually.
@Scope	Core	Bean class	Defines scope: singleton, prototype, request, session.
@Lazy	Core	Bean class	Delays bean creation until first use.
@PostConstruct	Lifecycle	Method	Runs after dependency injection.
@PreDestroy	Lifecycle	Method	Runs before bean is destroyed.
@Value	Config	Field	Injects value from properties file.
@PropertySource	Config	Config class	Loads external properties file.
@Profile	Config	Class/Method	Activates bean only for specific environment.
@ConfigurationProperties	Boot Config	Config class	Binds properties to Java object.
@RequestMapping	Web	Controller	Maps HTTP requests to methods.
@GetMapping	Web	Controller	Maps GET requests.
@PostMapping	Web	Controller	Maps POST requests.
@PutMapping	Web	Controller	Maps PUT requests.
@DeleteMapping	Web	Controller	Maps DELETE requests.
@PathVariable	Web	Method param	Reads value from URL path.
@RequestParam	Web	Method param	Reads query parameters.
@RequestBody	Web	Method param	Reads JSON body into object.

@ResponseBody	Web	Method	Converts return value to JSON.
@ControllerAdvice	Exception	Global	Handles exceptions globally.
@ExceptionHandler	Exception	Method	Handles specific exception.
@ResponseStatus	Exception	Exception class	Sets HTTP status for exception.
@Slf4j (Lombok)	Logging	Class	Provides logger without boilerplate.
@EnableScheduling	Boot	Config class	Enables scheduled tasks.
@Scheduled	Boot	Method	Runs method on schedule.
@ConditionalOnClass	Boot AutoConfig	Auto config	Loads bean only if class exists.
@ConditionalOnProperty	Boot AutoConfig	Auto config	Loads bean if property is present.
@ConditionalOnMissingBean	Boot AutoConfig	Auto config	Loads bean if no other exists.
@Order	Core	Bean class	Controls execution order of beans.
@Import	Core	Config class	Imports another configuration.